

Corrigé – Examen terminal: Programmation

L1 MPC1

28 mai 2026

Barème (total : 20 points)

Question	Contenu	Points
<i>Exercice 1 — Programmation orientée objet</i>		8,5
1.1.1	Attributs d'instance (liste + types)	0,5
1.1.2	Encapsulation () + méthode <code>est_disponible</code>	0,5
1.1.3	Méthode <code>emprunter</code>	0,5
1.1.4	Méthode <code>__str__</code>	0,5
1.2.1	<code>ajouter</code> + <code>livres_disponibles</code>	0,5
1.2.2	<code>chercher</code> + <code>emprunter</code> (Bibliothèque)	1,5
1.3.1	Définition de l'héritage	0,5
1.3.2	Classe <code>LivreNumérique</code> + <code>super()</code>	0,5
1.3.3	Surcharge <code>emprunter</code>	0,5
1.3.4	Surcharge <code>__str__</code>	0,5
1.3.5	Polymorphisme (justification)	1
1.4.1	Boîte UML de <code>Livre</code>	0,5
1.4.2	<code>LivreNumérique</code> + flèche d'héritage	0,5
1.4.3	<code>Bibliothèque</code> + relation avec <code>Livre</code> + multiplicité	0,5
<i>Exercice 2 — Séparation des responsabilités</i>		9,5
2.1.1	Classification M/V/C des 6 éléments (0,5 pt chacun)	3
2.1.2	Deux inconvénients du code monolithique	1
2.1.3	Parties à réécrire pour une interface web	1
2.2.1	Différence héritage / composition + exemples	1
2.2.2	Constructeur de <code>Application</code>	0,5
2.2.3	Méthode <code>emprunter</code> dans <code>Application</code>	0,5
2.3.1	Diagramme UML <code>Application/Bibliothèque</code> + nature de la relation	1,5
2.3.2	Créer vs recevoir les dépendances (composition vs association)	1
<i>Exercice 3 — Implémentation du design pattern MVC</i>		8
3.1.1	<code>VueTerminal</code> : 3 méthodes (0,5 pt chacune)	1,5
3.2.1	<code>Contrôleur</code> : <code>__init__</code> , <code>afficher_disponibles</code> , <code>emprunter</code>	2
3.2.2	<code>Contrôleur.lancer</code>	1
3.3.1	Code de lancement de l'application	0,5
3.3.2	Méthodes requises pour <code>VueGraphique</code> + principe de substitution	1
3.4.1	Diagramme UML complet MVC	1
3.4.2	Ajout <code>VueGraphique</code> + principe de substitution	1
Total		26

EXERCICE 1 — PROGRAMMATION ORIENTÉE OBJET

1.1 La classe Livre

Question 1.1.1 Qu'appelle-t-on un *attribut d'instance* ? Listez les attributs d'instance de la classe `Livre` et donnez leur type.

Réponse. Un attribut d'instance est une variable propre à chaque objet, créée dans `__init__` via `self`. Chaque instance possède sa propre copie indépendante de ces attributs. La classe `Livre` en possède trois :

- `titre : str`
- `auteur : str`
- `_disponible : bool`

Question 1.1.2 Pourquoi l'attribut `_disponible` est-il préfixé d'un tiret bas ? Ajoutez une méthode `est_disponible(self) -> bool`.

Réponse. Le tiret bas indique par convention que l'attribut est *protégé* : il ne doit pas être lu ni modifié directement depuis l'extérieur de la classe. On passe par des méthodes dédiées pour respecter l'encapsulation.

```
def est_disponible(self) -> bool:
    return self._disponible
```

Question 1.1.3 Complétez `emprunter(self) -> bool`.

Réponse.

```
def emprunter(self) -> bool:
    if self._disponible:
        self._disponible = False
        return True
    return False
```

Question 1.1.4 Écrivez `__str__(self) -> str`.

Réponse.

```
def __str__(self) -> str:
    etat = "disponible" if self._disponible else "emprunté"
    return f"{self.titre} ({self.auteur}) [{etat}] "
```

1.2 La classe Bibliothèque

Question 1.2.1 Ajoutez `ajouter` et `livres_disponibles`.

Réponse.

```
def ajouter(self, livre: Livre):
    self._livres.append(livre)

def livres_disponibles(self) -> list:
    return [l for l in self._livres if l.est_disponible()]
```

Question 1.2.2 Ajoutez `chercher` puis `emprunter`.

Réponse.

```
def chercher(self, titre: str):
    for livre in self._livres:
        if livre.titre == titre:
```

```

        return livre
    return None

def emprunter(self, titre: str) -> bool:
    livre = self.chercher(titre)
    if livre is None:
        return False
    return livre.emprunter()

```

1.3 Héritage : le livre numérique

Question 1.3.1 Qu'appelle-t-on l'héritage ?

Réponse. L'héritage permet à une classe *filles* de reprendre automatiquement les attributs et méthodes d'une classe *parente*. La classe fille est une spécialisation de la parente (relation « est-un ») : tout objet de la classe fille *est aussi* un objet de la classe parente. La classe fille peut ajouter de nouveaux membres ou *surcharger* (redéfinir) ceux hérités.

Question 1.3.2 Écrivez la classe `LivreNumérique` héritant de `Livre`.

Réponse.

```

class LivreNumérique(Livre):
    def __init__(self, titre: str, auteur: str):
        super().__init__(titre, auteur)

```

L'appel à `super().__init__()` délègue l'initialisation des attributs (`titre`, `auteur`, `_disponible`) à la classe parente.

Question 1.3.3 Surchargez `emprunter` pour qu'elle retourne toujours `True`.

Réponse.

```

def emprunter(self) -> bool:
    return True

```

On ne modifie pas `_disponible` : un livre numérique est toujours disponible, son état ne change jamais.

Question 1.3.4 Surchargez `__str__`.

Réponse.

```

def __str__(self) -> str:
    return f"{self.titre} ({self.auteur}) [numérique]"

```

Question 1.3.5 Peut-on ajouter un `LivreNumérique` à une `Bibliothèque` ? Quel principe illustre ce comportement ?

Réponse. Oui. `Bibliothèque.ajouter` attend un `Livre` ; or `LivreNumérique` hérite de `Livre`, donc tout `LivreNumérique` *est* un `Livre` et peut le remplacer partout. Lorsque `Bibliothèque.emprunter` appellera `livre.emprunter()`, Python exécutera la version surchargée de `LivreNumérique`, qui retourne toujours `True`. Ce mécanisme s'appelle le **polymorphisme** : un même appel de méthode produit un comportement différent selon le type réel de l'objet.

1.4 Modélisation UML

Question 1.4.1 Dessinez la boîte UML de la classe `Livre`.

Réponse.

```

+-----+
|               Livre               |
+-----+

```

```

| - titre : str          |
| - auteur : str         |
| - _disponible : bool   |
+-----+
| + est_disponible() : bool |
| + emprunter() : bool     |
| + __str__() : str        |
| + __init__(titre, auteur) |
+-----+

```

Question 1.4.2 Ajoutez LivreNumérique au diagramme.

Réponse.

```

+-----+
|      LivreNumérique      |
+-----+
|                          |
+-----+
| + emprunter() : bool     |
| + __str__() : str        |
+-----+
      (héritage)
      |
+-----+
|      Livre               |
+-----+
| ...                      |
+-----+

```

La boîte de LivreNumérique ne répète pas les membres hérités; on n'y inscrit que les méthodes surchargées. L'attribut `_disponible` n'apparaît pas car aucun attribut nouveau n'est ajouté.

Question 1.4.3 Ajoutez Bibliothèque au diagramme.

Réponse.

```

+-----+          1..*   +-----+
|      Bibliothèque      | -----> | Livre |
+-----+          +-----+
| - _livres : list       |
+-----+
| + ajouter(livre)       |
| + chercher(titre) : Livre |
| + emprunter(titre) : bool |
| + livres_disponibles() : list |
+-----+

```

La relation est une **composition** : une Bibliothèque contient (et gère la durée de vie de) ses livres. La multiplicité `1..*` indique qu'une bibliothèque peut contenir un ou plusieurs livres (on peut aussi écrire `0..*` si une bibliothèque vide est admise).

EXERCICE 2 – SÉPARATION DES RESPONSABILITÉS

2.1 Analyse

Question 2.1.1 Classifiez les éléments (a) à (f) selon M, V ou C.

Réponse.

- (a) `choix = input(...)` → **Vue** : c'est une saisie utilisateur.
- (b) `dispo = [l for l in livres if l["dispo"]]` → **Modèle** : filtrage des données métier.
- (c) `print(f"- {l['titre']})...")` → **Vue** : affichage d'information à l'écran.
- (d) `titre = input("Titre : ")` → **Vue** : saisie utilisateur.
- (e) `livre = next(...)` → **Modèle** : recherche dans les données.
- (f) `livre["dispo"] = False` → **Modèle** : modification de l'état d'un livre.

Note : les blocs `if/elif` qui décident quelle action effectuer selon `choix` relèvent du **Contrôleur**.

Question 2.1.2 Donnez deux inconvénients du code monolithique.

Réponse.

1. **Non réutilisabilité** : impossible de changer d'interface (terminal, web, graphique) sans réécrire la totalité de la fonction, y compris la logique métier.
2. **Difficulté de maintenance** : toute correction ou évolution oblige à lire l'intégralité du code mélangé, avec un risque fort d'introduire des régressions dans des parties sans lien avec la modification.

Question 2.1.3 Que faudrait-il réécrire pour passer à une interface web ?

Réponse. Toutes les parties **Vue** : les `print(...)` (remplacés par du HTML/JSON) et les `input(...)` (remplacés par des requêtes HTTP). Dans une architecture MVC correcte, seule la Vue change ; le Modèle (règles métier, données) et le Contrôleur (logique de décision) restent intacts.

2.2 Refactoring par composition

Question 2.2.1 Différence entre héritage et composition.

Réponse.

- **Héritage** (relation « est-un ») : `LivreNumérique` est un `Livre`, il hérite de ses attributs et méthodes. On l'utilise quand la classe fille est une spécialisation de la parente.
- **Composition** (relation « a-un ») : `Bibliothèque` contient des `Livre`, elle ne les étend pas. On l'utilise quand un objet est composé d'autres objets sans en être une variante.

Question 2.2.2 Constructeur de `Application` par composition.

Réponse.

```
class Application:
    def __init__(self):
        self._bibliotheque = Bibliothèque()
```

`Application` ne hérite pas de `Bibliothèque` ; elle en *possède* une instance. C'est la composition.

Question 2.2.3 Méthode `emprunter(self)` de `Application`.

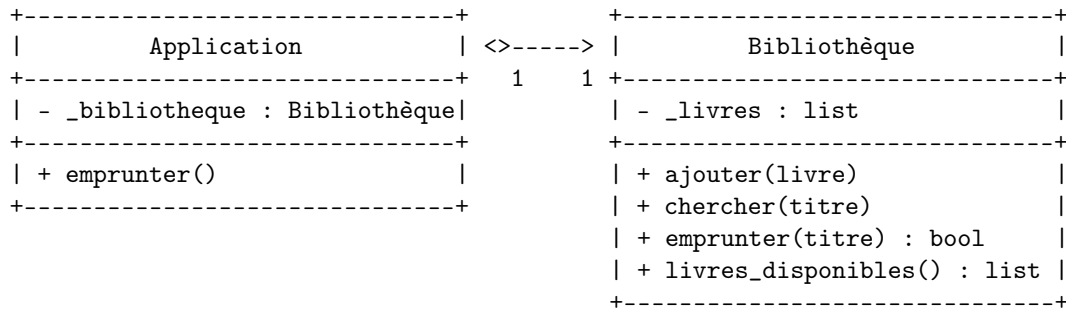
Réponse.

```
def emprunter(self):
    titre = input("Titre : ")
    succes = self._bibliotheque.emprunter(titre)
    if succes:
        print("Emprunté.")
    else:
        print("Indisponible ou introuvable.")
```

La logique métier (l'emprunt) est maintenant déléguée à `Bibliothèque` ; la méthode ne manipule plus directement les données.

2.3 Modélisation UML

Question 2.3.1 Dessinez le diagramme de classes UML de **Application** ainsi que son lien avec **Bibliothèque**.
Réponse.



La relation est une **composition** (losange plein, <>) car **Application** crée la **Bibliothèque** dans son constructeur et en est propriétaire : si l'application disparaît, la bibliothèque disparaît avec elle. La multiplicité 1 : 1 signifie qu'une **Application** possède exactement une **Bibliothèque**. Une simple **association** (flèche sans losange) s'utiliserait si les deux objets avaient des cycles de vie indépendants.

Question 2.3.2 Créer vs recevoir les dépendances en UML.

Réponse.

- **Contrôleur crée ses dépendances** : relation de **composition** (losange plein) de **Contrôleur** vers **Bibliothèque** et vers **VueTerminal**. Le **Contrôleur** possède ces objets.
- **Contrôleur reçoit ses dépendances** : relation d'**association** simple (flèche sans losange). Les objets existent indépendamment du contrôleur.

Le second cas est préférable pour la testabilité : on peut passer au contrôleur une fausse vue ou un faux modèle (*mock*) lors des tests unitaires, sans modifier le code du contrôleur.

EXERCICE 3 — IMPLÉMENTATION DU *DESIGN PATTERN* MVC

3.1 La Vue

Question 3.1.1 Implémentez en Python la classe **VueTerminal**.

Réponse.

```

class VueTerminal:
    def afficher_livres(self, livres: list):
        if not livres:
            print("Aucun livre disponible.")
        else:
            for livre in livres:
                print(str(livre))

    def afficher_message(self, message: str):
        print(message)

    def demander_titre(self) -> str:
        return input("Titre : ")

```

La Vue ne contient aucune logique métier : elle se contente d'afficher et de transmettre les saisies.

3.2 Le Contrôleur

Question 3.2.1 Implémentez `__init__`, `afficher_disponibles` et `emprunter`.

Réponse.

```

class Contrôleur:
    def __init__(self, modele: Bibliothèque, vue: VueTerminal):
        self._modele = modele
        self._vue = vue

    def afficher_disponibles(self):
        livres = self._modele.livres_disponibles()
        self._vue.afficher_livres(livres)

    def emprunter(self):
        titre = self._vue.demander_titre()
        succes = self._modele.emprunter(titre)
        if succes:
            self._vue.afficher_message("Emprunté avec succès.")
        else:
            self._vue.afficher_message("Indisponible ou introuvable.")

```

Le Contrôleur ne fait jamais d’affichage direct (pas de print) ni d’accès direct aux données : il délègue toujours à la Vue ou au Modèle.

Question 3.2.2 Implémentez lancer.

Réponse.

```

def lancer(self):
    while True:
        print("1. Livres disponibles  2. Emprunter  0. Quitter")
        choix = input("Votre choix : ")
        if choix == "1":
            self.afficher_disponibles()
        elif choix == "2":
            self.emprunter()
        elif choix == "0":
            break

```

On pourrait déplacer le print et l’input du menu dans la Vue (méthodes `afficher_menu` et `demander_choix`) pour une séparation encore plus stricte.

3.3 Extensibilité

Question 3.3.1 Écrivez le code de lancement de l’application.

Réponse.

```

modele = Bibliothèque()
modele.ajouter(Livre("Le Petit Prince", "Antoine de Saint-Exupéry"))
modele.ajouter(Livre("1984", "George Orwell"))

vue = VueTerminal()
contrôleur = Contrôleur(modele, vue)
contrôleur.lancer()

```

Question 3.3.2 Quelles méthodes `VueGraphique` doit-elle posséder ?

Réponse. `VueGraphique` doit exposer exactement les mêmes méthodes que celles appelées par `Contrôleur` sur la vue :

- `afficher_livres(self, livres: list)`
- `afficher_message(self, message: str)`
- `demander_titre(self) -> str`

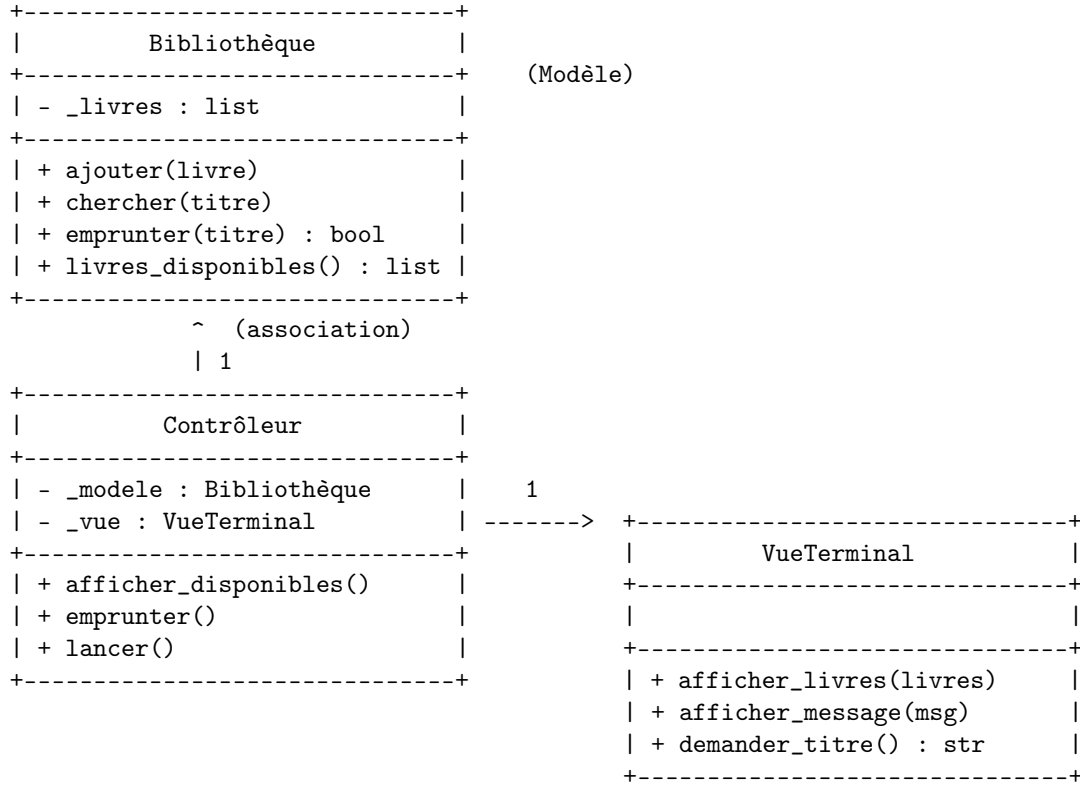
C’est le principe de **substitution** : `VueGraphique` peut remplacer `VueTerminal` partout sans que `Contrôleur` ne soit modifié, car il n’interagit avec la vue qu’au travers de ces trois méthodes. Pour passer à une interface graphique,

seule la ligne `vue = VueTerminal()` changerait en `vue = VueGraphique()` ; le reste du code de lancement est inchangé.

3.4 Modélisation UML

Question 3.4.1 Diagramme de classes complet de l'architecture MVC.

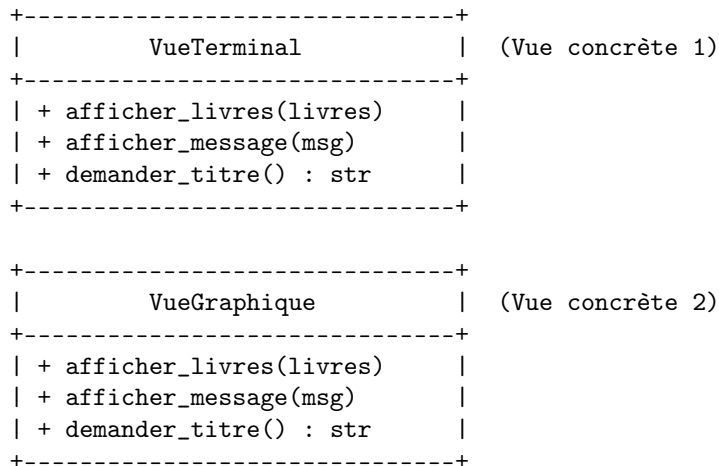
Réponse.



Contrôleur est associé à Bibliothèque et à VueTerminal (associations simples, sans losange : il reçoit ses dépendances en paramètre). La multiplicité est 1 des deux côtés : un contrôleur utilise exactement un modèle et une vue.

Question 3.4.2 Ajout de VueGraphique et principe de substitution.

Réponse.



Contrôleur -----> VueTerminal (ou VueGraphique)

VueGraphique n'hérite pas de **VueTerminal** : ce sont deux classes indépendantes qui exposent la même *interface* (mêmes signatures de méthodes). En UML strict on modéliserait une interface **IVue** dont les deux héritent, mais en Python le typage canard suffit.

Le diagramme illustre le **principe de substitution** : **Contrôleur** ne connaît que les méthodes **afficher_livres**, **afficher_message** et **demander_titre**. N'importe quel objet proposant ces trois méthodes peut remplacer la vue sans modifier **Contrôleur**.